

CampusFind

Detailed Development Evidence and Screenshot Briefing

ICT205 Mobile Application Development

This document explains the development of the CampusFind mobile application using screenshots captured during implementation, Firebase integration, feature testing and UI refinement.

Project	CampusFind - Lost & Found Campus App
Platform	Flutter / Android
Backend	Firebase Authentication + Cloud Firestore
Device features	Camera / Gallery + GPS Location

1. Initial Flutter Project Setup

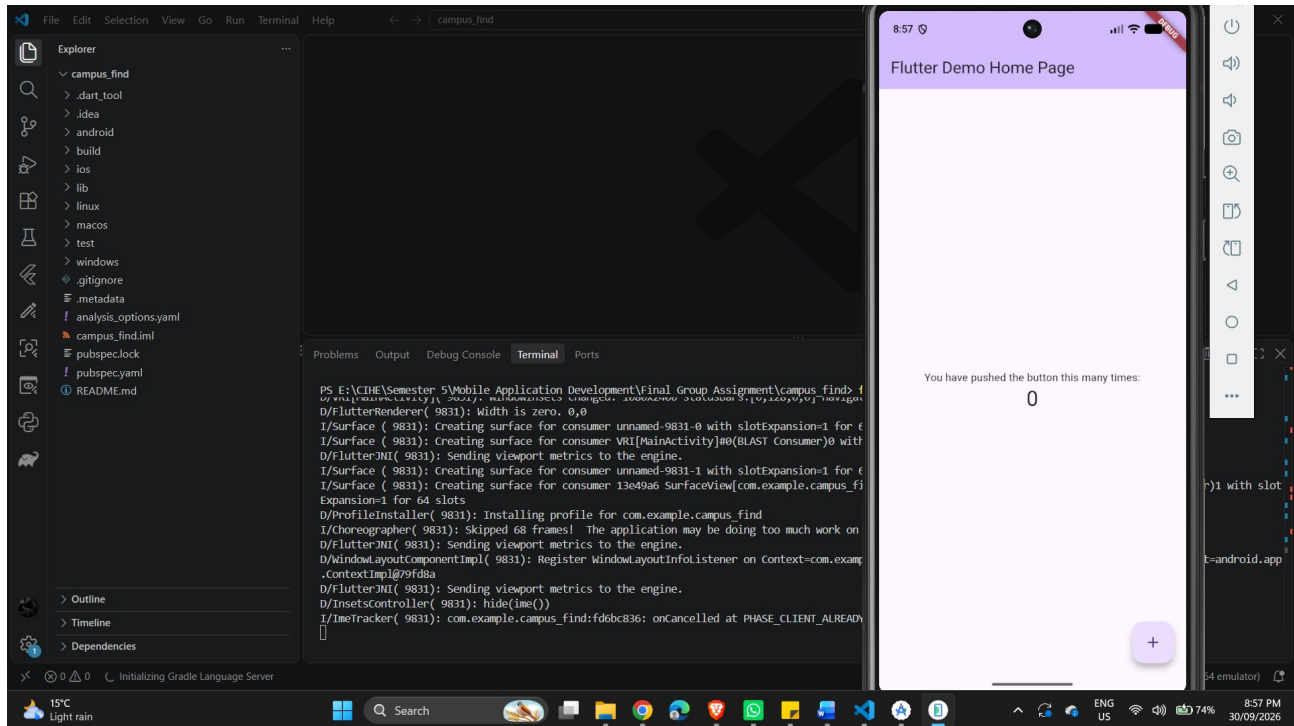


Figure 1. Default Flutter project running successfully on the Android emulator.

The project was first created and opened in Visual Studio Code. The default Flutter counter application was launched on the Android emulator to verify that the Flutter SDK, project dependencies, emulator and development environment were working correctly. This provided a clean starting point before the application was customised for the CampusFind project.

2. First CampusFind Interface

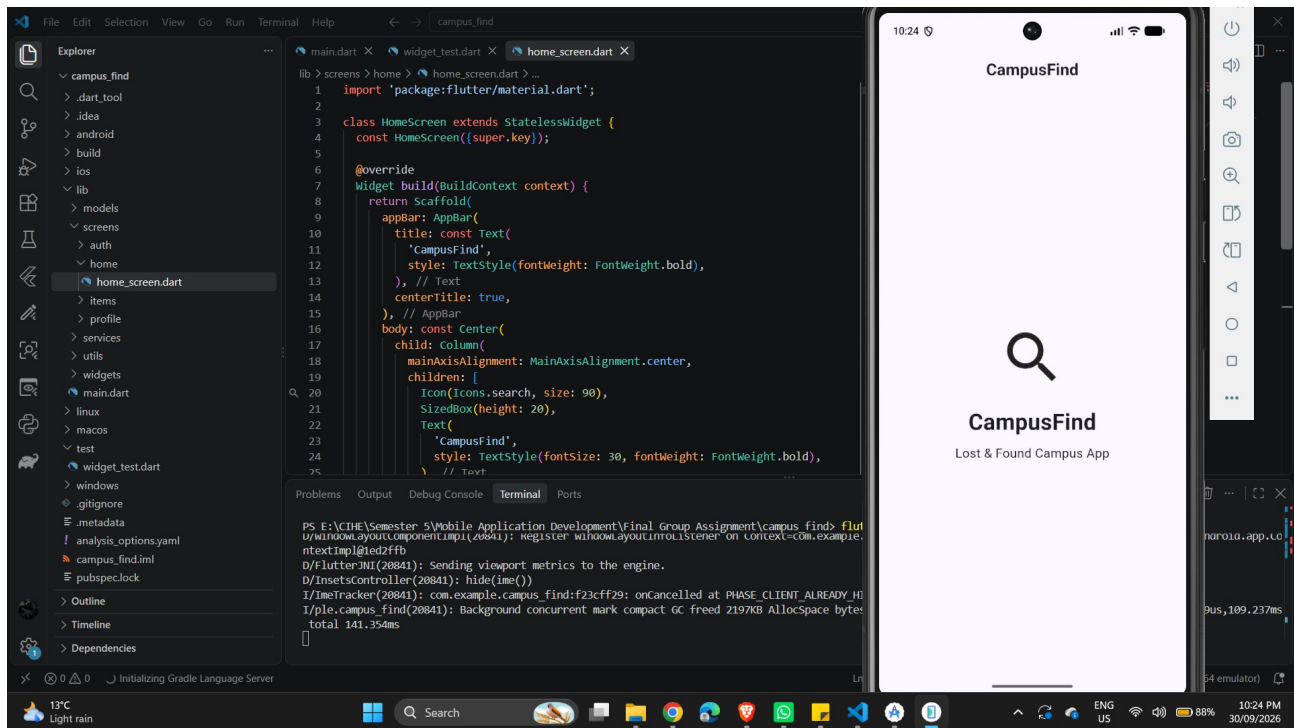


Figure 2. First customised CampusFind interface replacing the default Flutter starter screen.

After the development environment was verified, the default Flutter interface was replaced with an initial CampusFind screen. The project name, search icon and Lost & Found Campus App description established the purpose of the application and confirmed that the custom Flutter interface was running successfully.

3. Firebase Authentication Setup

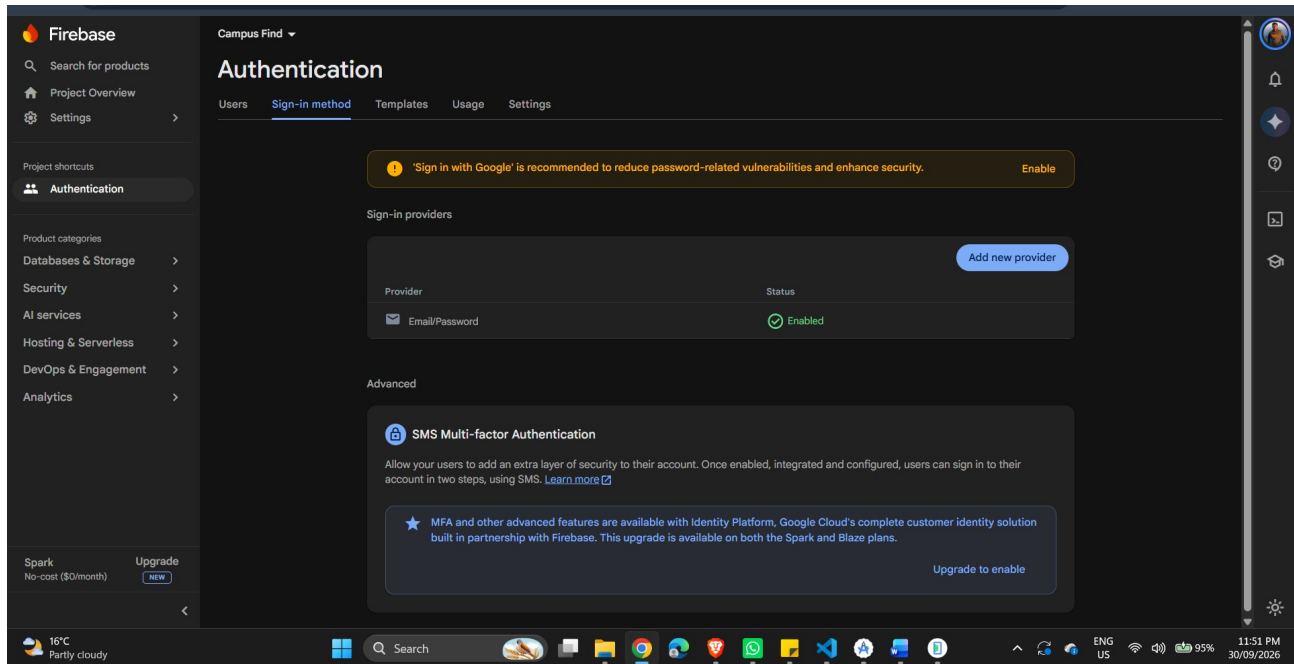


Figure 3. Firebase Authentication enabled for the CampusFind project.

Firebase was configured as the backend platform for the application. Authentication was enabled to support secure user access. This stage established the connection between the mobile application and Firebase services and prepared the project for email/password and Google authentication.

- Remote authentication service configured in Firebase.
- Foundation created for user registration and login.
- Backend connection prepared for later Firestore integration.

4. Login Interface and Validation

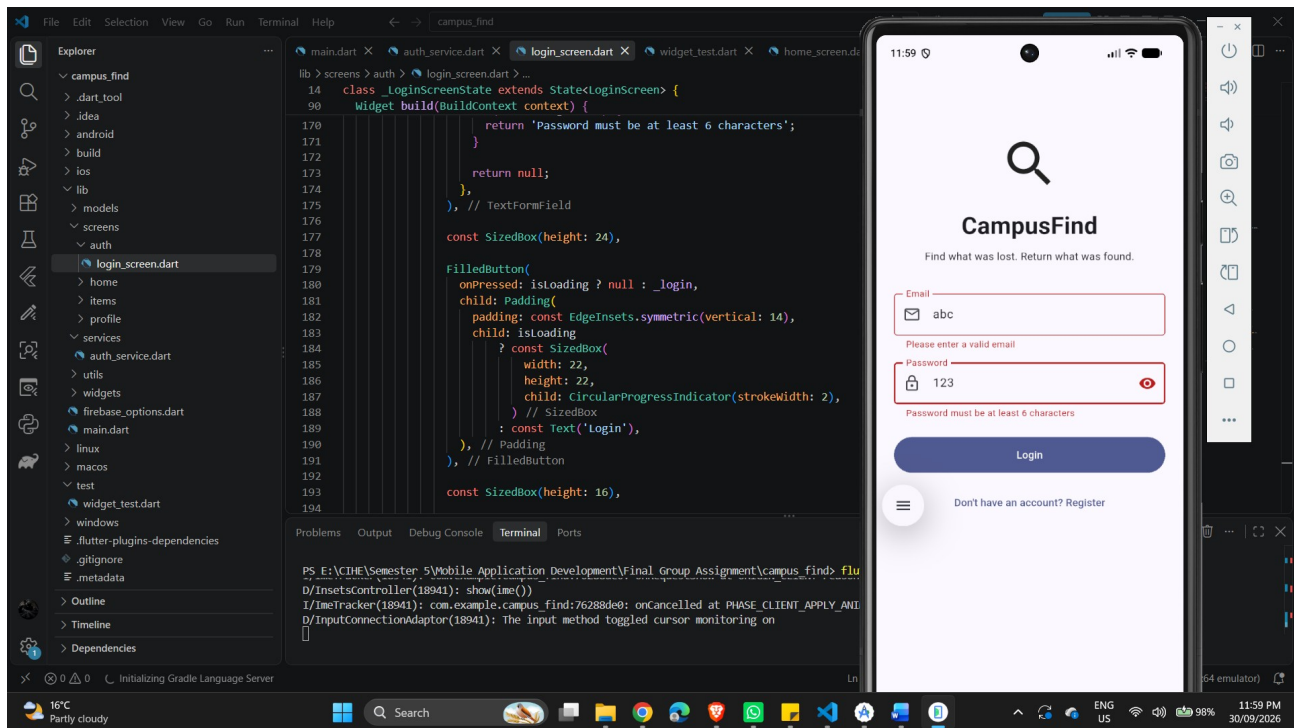


Figure 4. CampusFind Login screen showing form validation.

A custom Login screen was created with email and password fields, a login button and navigation to account registration. Validation was implemented so incomplete or invalid input produces clear feedback instead of allowing an incorrect request to continue. This improves usability and helps prevent invalid authentication attempts.

5. User Registration

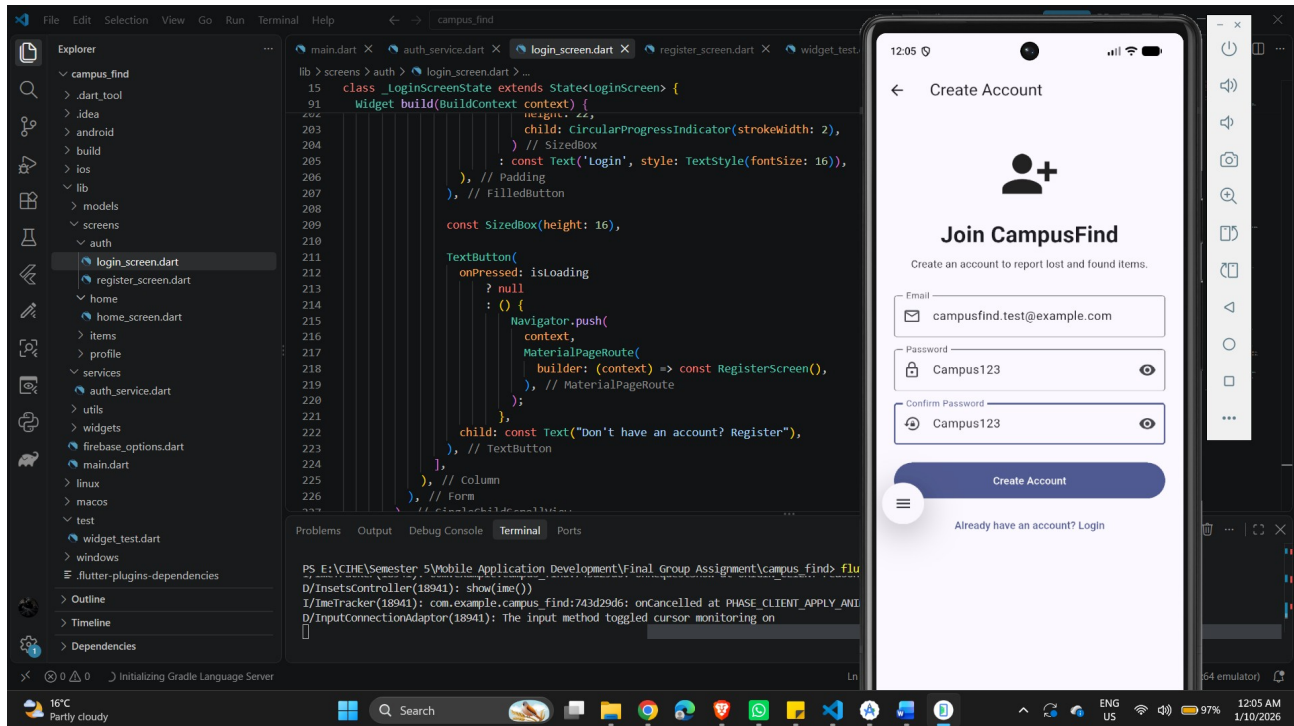


Figure 5. Create Account screen used for new CampusFind users.

The registration screen allows a new user to create an account by entering an email address, password and password confirmation. The registration flow was connected to Firebase Authentication so successfully registered accounts could be stored and later used to access the application.

6. Firestore Database Configuration

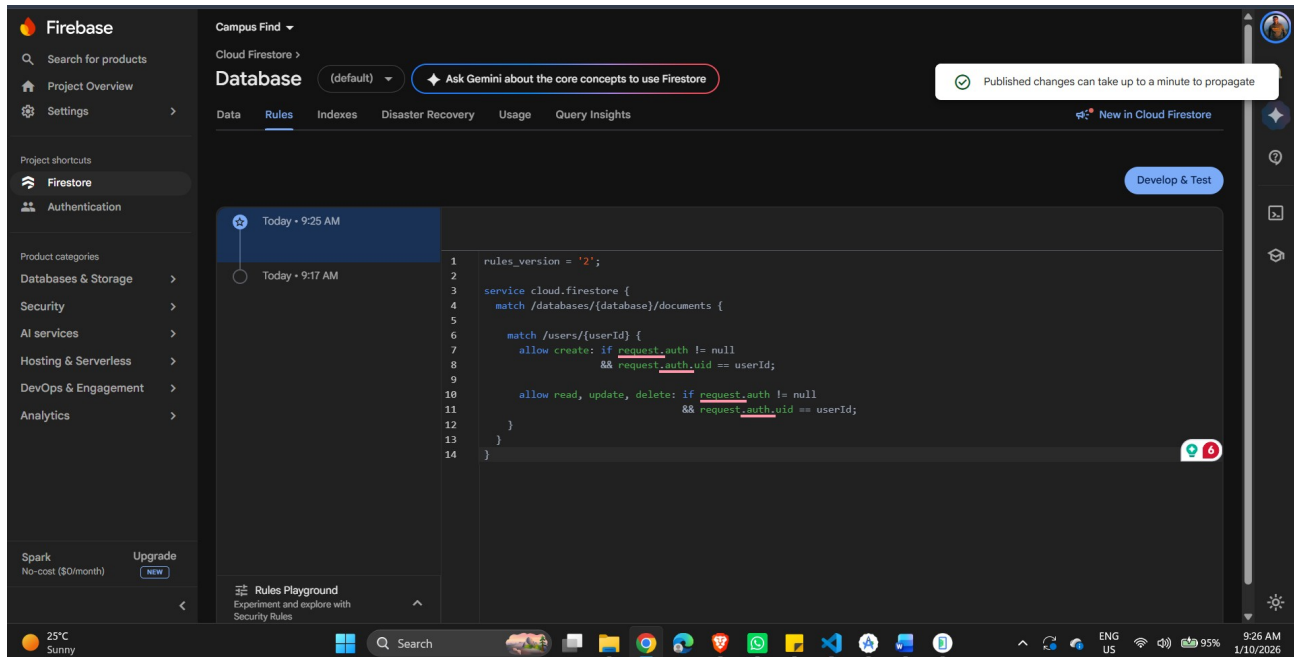


Figure 6. Cloud Firestore database configured for CampusFind.

Cloud Firestore was added as the remote database for application data. This moved the project beyond local or hard-coded content and allowed data to be written to and read from a backend service. Firestore became the main storage layer for user profiles and Lost/Found item reports.

7. Home Screen and Main Navigation

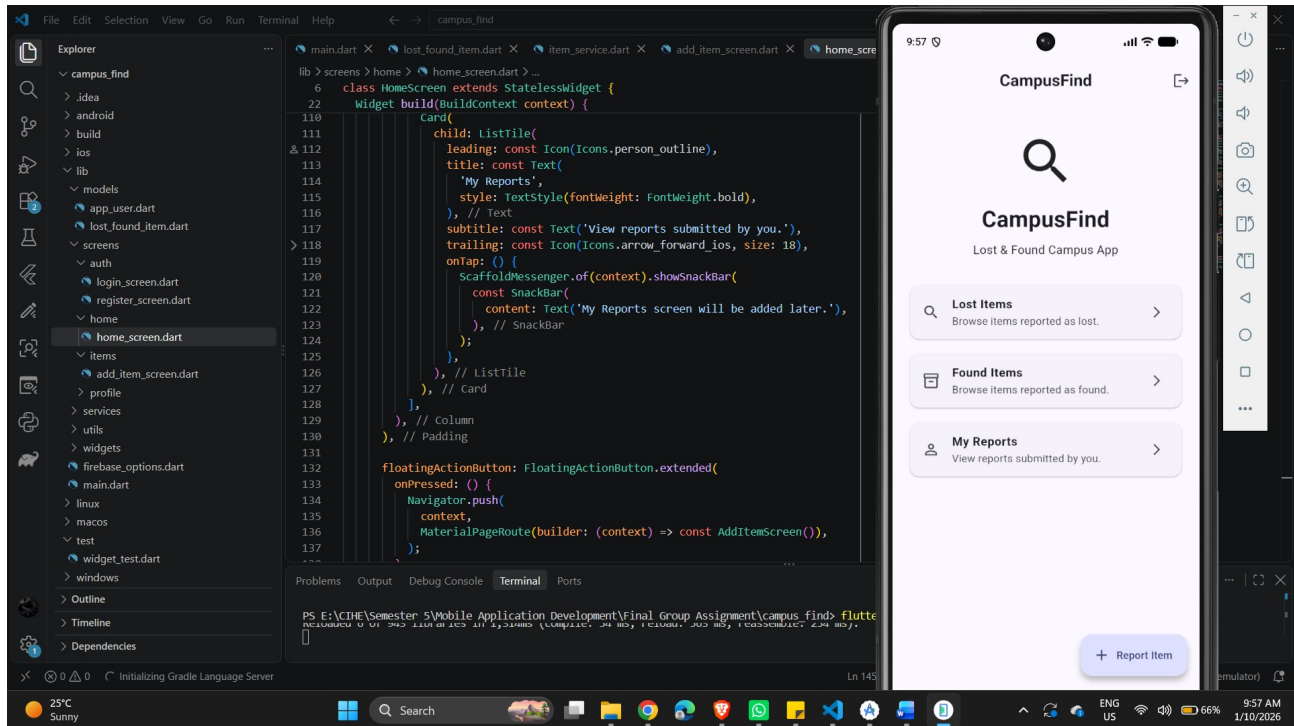


Figure 7. Early CampusFind Home screen with access to Lost Items, Found Items, My Reports and Report Item.

The Home screen was developed as the central navigation point of the application. Users can browse Lost Items, browse Found Items, view their own reports or open the report form. The structure provides a simple user flow and makes the main application functions accessible from one screen.

8. Lost and Found Report Form

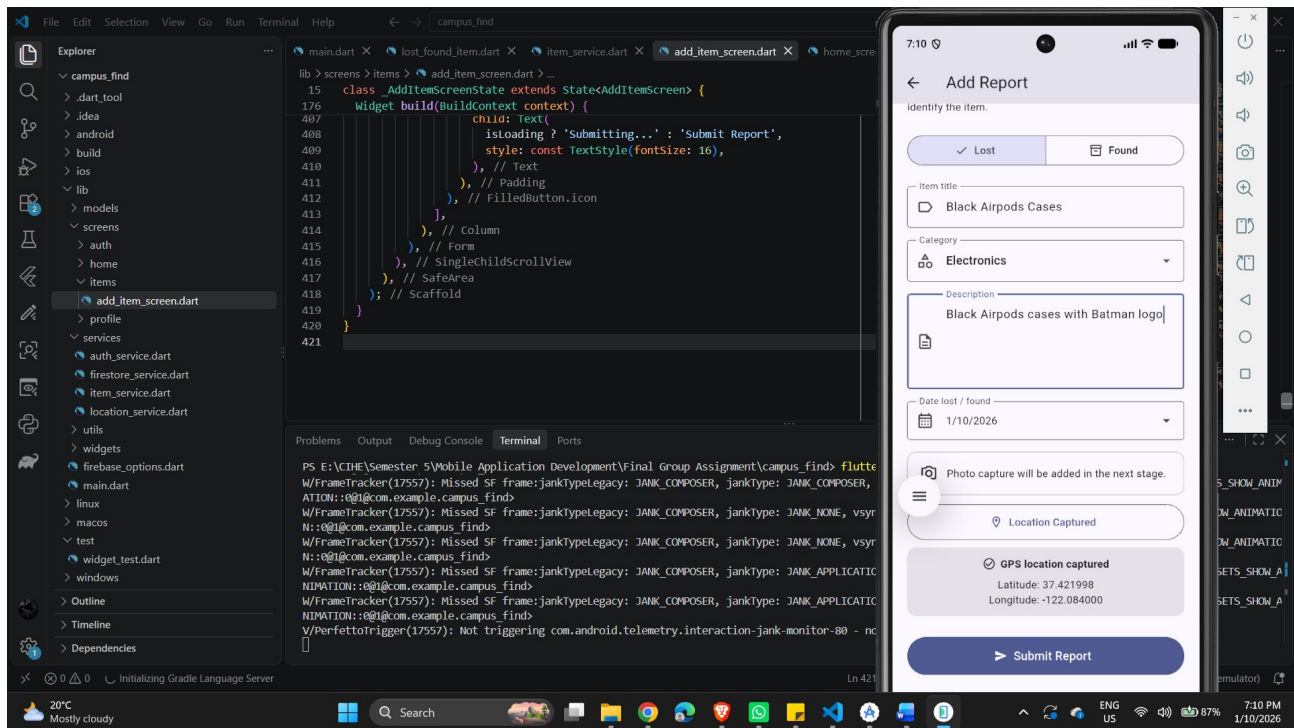


Figure 8. Add Report screen containing report type, item details, date and device options.

The Add Report screen was created so an authenticated user can submit either a Lost or Found report. The form collects the item title, category, description and date. It also contains options for adding a photo and capturing the item location, making the report more informative and useful for other users.

9. GPS Location Integration

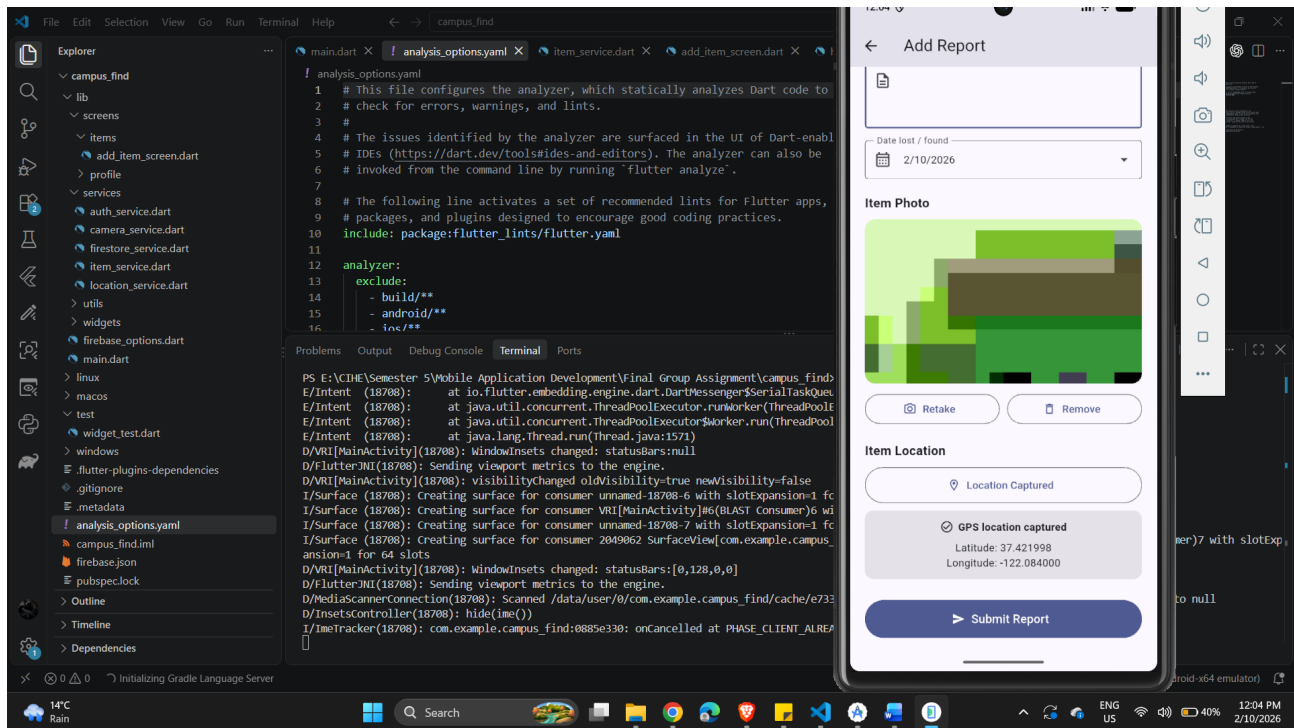


Figure 9. Add Report screen showing a captured item photo and GPS coordinates.

GPS/location services were integrated as a meaningful mobile-device feature. The application requests location access and captures latitude and longitude for the report. The captured coordinates are displayed to the user before submission and are stored with the report in Firestore. This helps describe where an item was lost or found.

10. Photo Selection and Preview

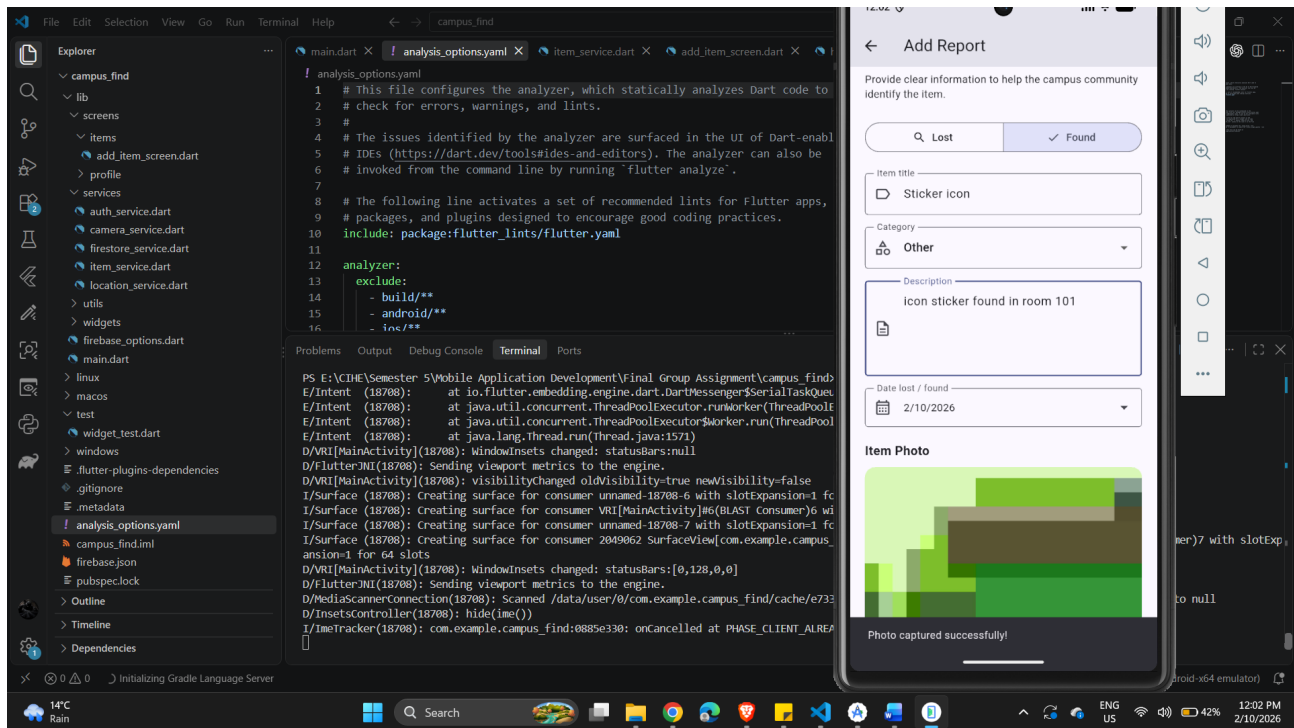


Figure 10. Item photo displayed within the report form before submission.

Camera and gallery functionality was integrated into the report workflow. A user can select a photograph and preview it before submitting the report. The final application also supports taking a new photo, selecting from the gallery, changing the selected image and removing it. In the current prototype, the image is previewed locally; persistent Firebase Storage upload remains a future improvement.

11. Report Stored in Firestore

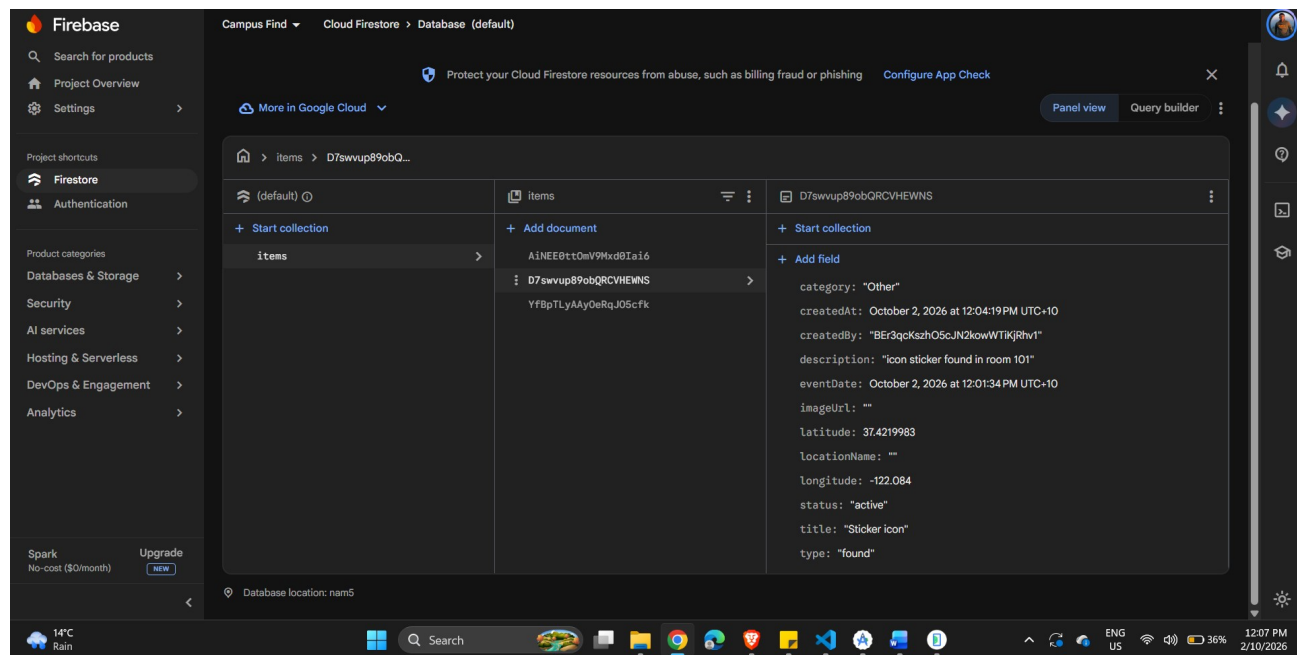


Figure 11. Firestore document created after a report was submitted from the application.

After the user completes the report form and selects Submit Report, the application validates the information and writes the report to Cloud Firestore. The database evidence shows that data created in Flutter is successfully persisted remotely. The stored information includes item details, report type, status, creator information and location data.

12. Lost and Found Item Lists

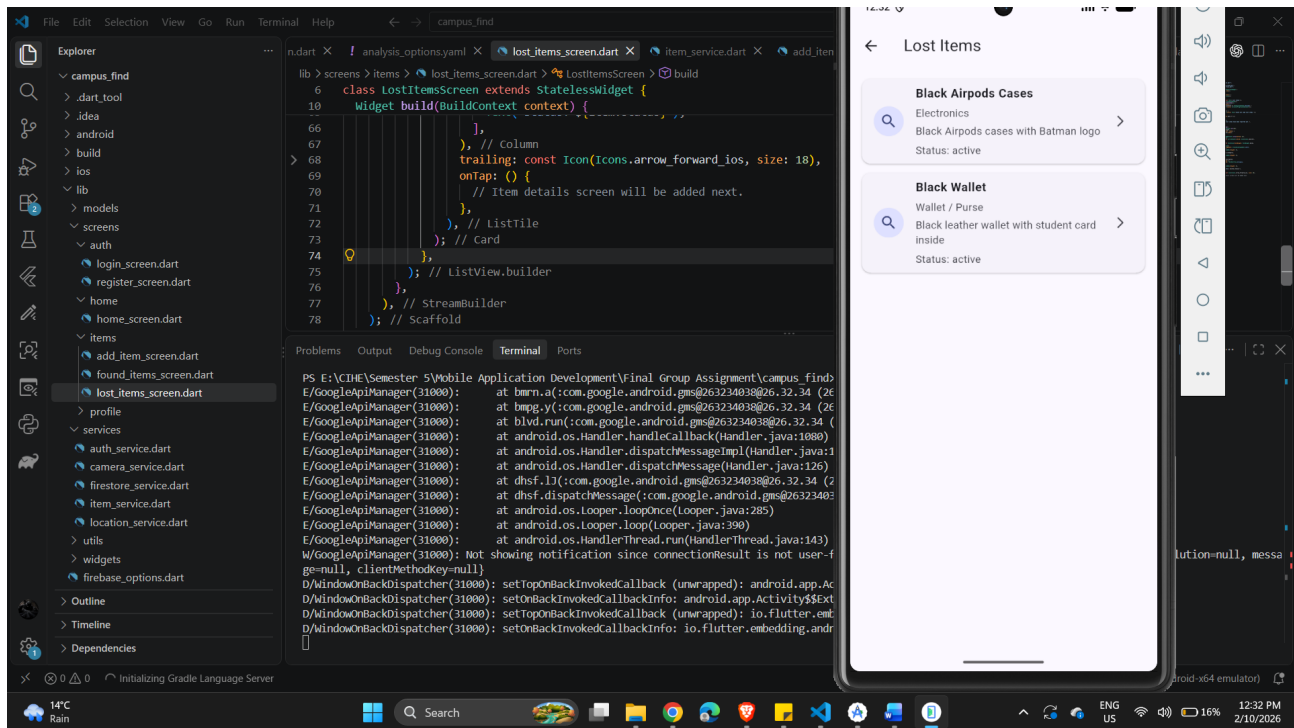


Figure 12. Lost Items screen displaying reports retrieved from Firestore.

Separate Lost Items and Found Items screens were developed to retrieve and display report data from Firestore. Item cards provide a quick summary and can be selected to view additional information. This demonstrates backend read operations and turns the stored Firestore documents into useful application content.

13. Item Details and Owner Actions

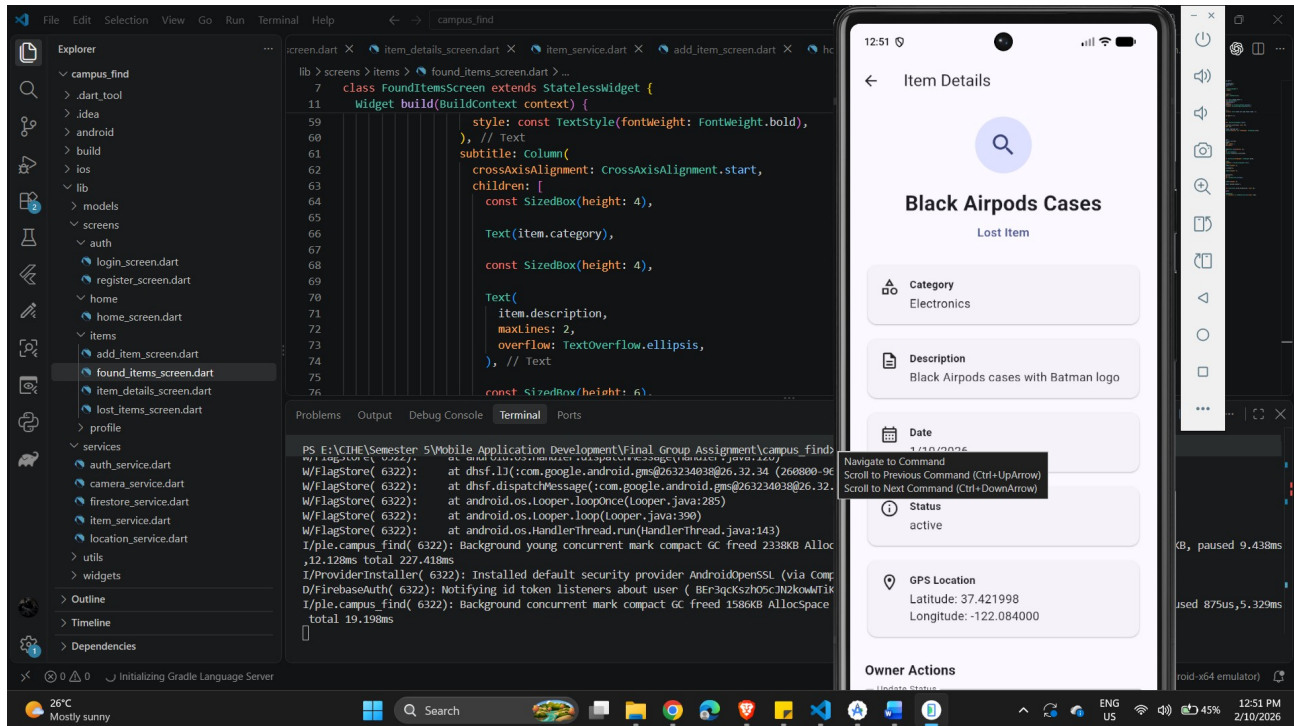


Figure 13. Item Details screen for a Black AirPods Case report.

The Item Details screen displays the complete information for a selected report, including title, type, category, description, date, status and GPS coordinates. When the authenticated user owns the report, owner actions can be provided for updating the status or deleting the report. Ownership is determined using the Firebase UID stored with the report.

14. My Reports

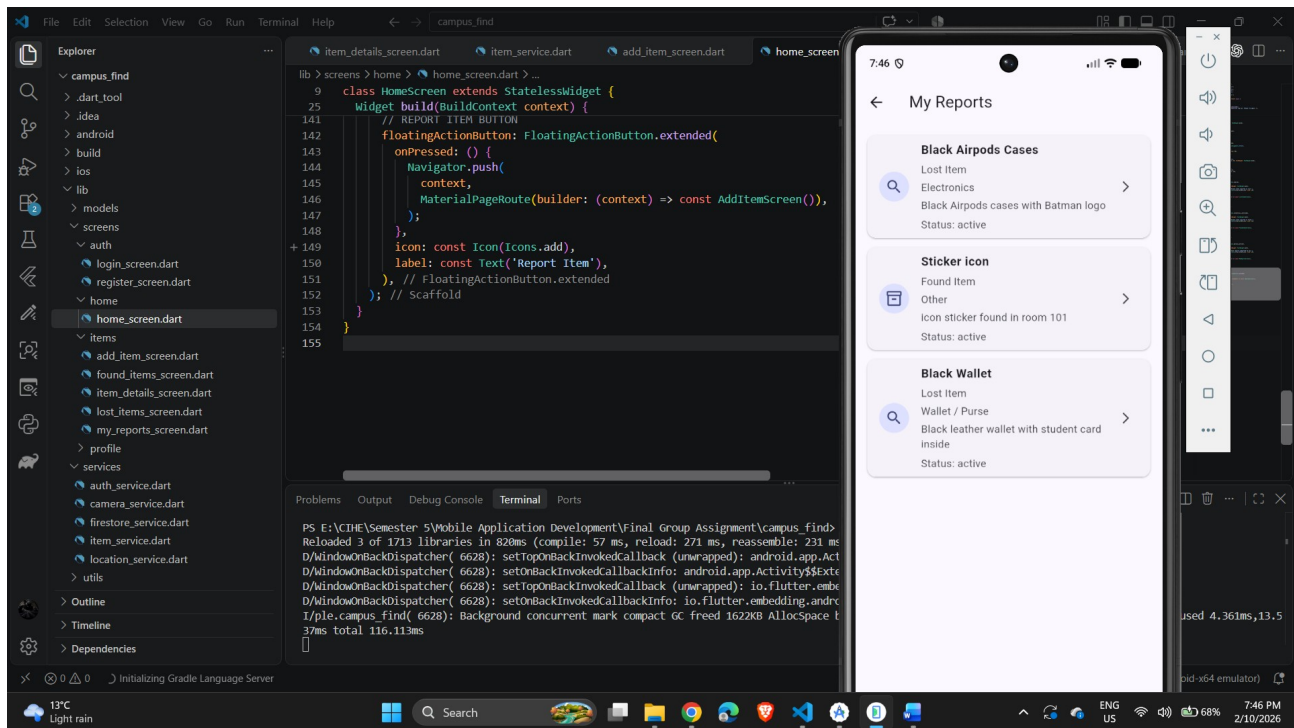


Figure 14. My Reports screen showing reports created by the authenticated user.

The My Reports screen filters the database using the currently authenticated user ID. This enables each user to manage the reports they personally created without mixing them with reports owned by other users. The screen provides a convenient location for reviewing Lost and Found reports and their current statuses.

15. Final UI/UX Upgrade

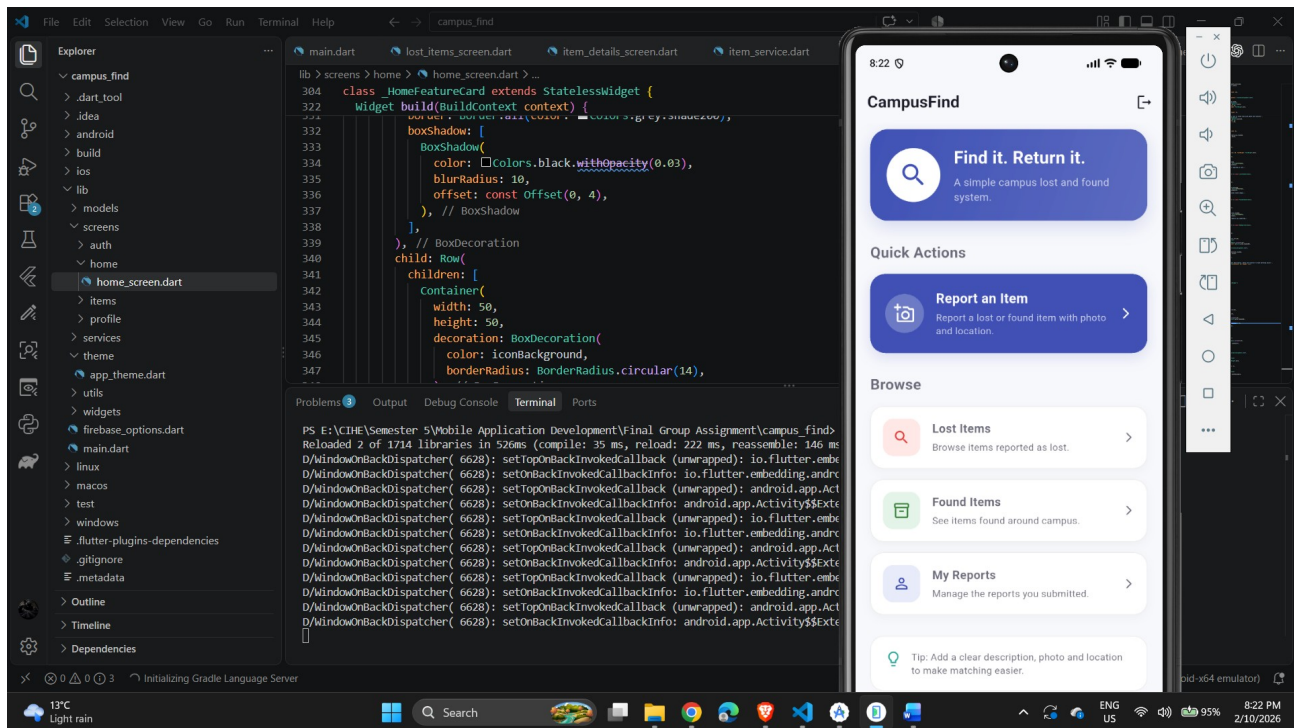


Figure 15. Redesigned CampusFind Home screen with improved visual hierarchy and quick actions.

The interface was later upgraded to improve usability and presentation quality. The redesigned Home screen introduces stronger CampusFind branding, a Find it. Return it. hero section, a prominent Report an Item quick action, clearer Browse cards, improved spacing and more consistent icons and colours. This stage demonstrates refinement of the application after the main functionality was completed.

16. Final Login Design

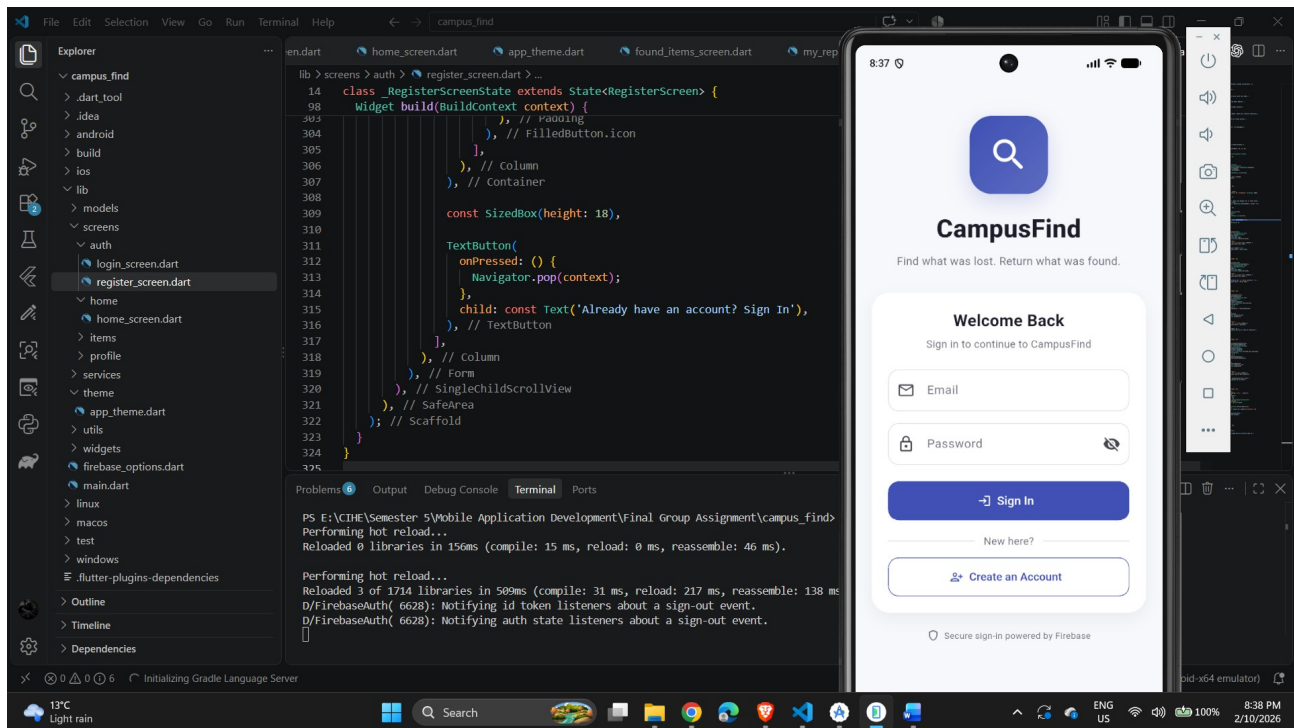


Figure 16. Final CampusFind Login screen after UI/UX refinement.

The Login screen was also redesigned to match the final CampusFind visual style. The updated screen uses stronger branding, improved spacing, a clearer card-based form and a more prominent sign-in action. Together with the redesigned Home screen, this provides a consistent and more professional final user experience.

Development Summary

The screenshot evidence demonstrates a progressive development process: the project began with a standard Flutter application, was converted into CampusFind, connected to Firebase, extended with authentication and Firestore, enhanced with reporting, GPS and photo features, and finally refined through UI/UX improvements. The evidence therefore documents both the technical implementation and the evolution of the application from an initial prototype to a functional release-ready prototype.